МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Ижевский государственный технический университет имени М.Т. Калашникова»

Кафедра «Программное обеспечение»

ОТЧЁТ
по лабораторной работе № 2
Архитектура и интерактивный прототип в коде
по дисциплине
«Проектирование и конструирование ПО»
Проект «Fit Nutrition Supply»

Выполнили студенты группы Б23-191-2: _____ подпись, дата Чипига А. А.

_____ подпись, дата Фазлыев А. А.

Проверил _____ подпись, дата Еланцев М. О.

Ижевск, 2026

СОДЕРЖАНИЕ

Введение..... 3
1 Уточнение концепции после замечаний..... 4
2 Структурная схема и технологии..... 5
3 Сценарии использования и интерфейс..... 6
4 Модель данных..... 10
5 Программные интерфейсы..... 13
6 Архитектурные решения..... 19
7 Проверка прототипа и покрытие требований..... 29
8 Работа с ИИ агентом..... 31
9 UX-доработки после тестирования..... 33
Заключение..... 36
Источники..... 37
Приложение А Схемы..... 38
Приложение Б Протокол проверки..... 50

ВВЕДЕНИЕ

Цель работы - спроектировать архитектуру Fit Nutrition Supply и создать исполняемый прототип с экранными формами и навигацией. Работу выполняют Фазлыев А. А. и Чипига А. А., группа Б23-191-2. Основание требований - сданный отчёт ЛР1, сохранённый в sources/submitted-lab1/. Номера требований в данном документе относятся именно к нему.

Прототип реализует связанный сценарий: параметры питания → рацион → рецепт и замена → список покупок → сравнение корзин. Это первый срез продукта, а не завершённая реализация всех требований ЛР1. Реальные интеграции магазинов и внешний ИИ не подключены. Учебные предложения пяти сетей и приближённые значения пищевой ценности явно обозначены в интерфейсе.

1 УТОЧНЕНИЕ КОНЦЕПЦИИ ПОСЛЕ ЗАМЕЧАНИЙ

Основная целевая аудитория - взрослые люди, заинтересованные в снижении веса и планирующие готовить самостоятельно. Приложение помогает связать калорийность меню с повседневными покупками. Возможность использования людьми с другими целями не меняет основную ЦА. Ижевск - город пилота, а не признак принадлежности к аудитории.

Бюджет и свободное время учитываются как ограничения: пользователь задаёт сумму на период и максимальное время приготовления блюда. Сервис не гарантирует, что любое сочетание ограничений выполнимо. При нехватке рецептов он сообщает об этом и сохраняет предыдущий план; при превышении бюджета показывает сумму превышения.

В дополнении к ЛР1 приведены таблицы аналогов с колонкой собственного продукта и уточнённое NFR-05 с фиксированными версиями браузеров. Сданные материалы сохранены без перезаписи, чтобы было видно исходное состояние и внесённые уточнения.

2 СТРУКТУРНАЯ СХЕМА И ТЕХНОЛОГИИ

Архитектура - модульный монолит. Браузер обращается к HTTP API; сервер проверяет сессию и ввод, вызывает расчётные функции и сохраняет данные в SQLite.

См. рисунок 1 - Структурная схема системы (приложение А).

Таблица 1

Подсистема	Ответственность	Реализация
UI	Общий каркас, профиль, семья, меню и закупки	app.js, flow.js, profile.js, family.js, result.js
HTTP API	JSON, валидация запросов, принадлежность данных пользователю	server.py, Handler
Авторизация	Регистрация, bcrypt, cookie-сессии и ограничение попыток	server.py, таблицы users, sessions, login_attempts
Профили	Дата рождения, готовность, семейные участники и нормы	profiles.py, family.py
Планирование	Норма калорий, подбор допустимых блюд, размеры порций	domain.py, validate_settings, generate_plan
Замена блюда	Фильтр кандидатов, прогноз корзины и цена ингредиентов	replacements.py, replacement_candidates, preview_replacements, meal_cost
Закупки	Список, запасы, упаковки, полнота, сравнение, бюджет	domain.py, shopping, calculate_baskets
Источник цен	Чтение предложений и обновление демонстрационного набора	prices.py, DemoPriceProvider
Хранение	Параметризованный SQL, транзакции, внешние ключи	server.py, connect, schema.sql

Онбординг связывает flow.js с profiles.py, замена - result.js с replacements.py. Handler передаёт им данные из SQLite. Расчёты используют domain.py без доступа к БД; SQL и авторизация находятся в server.py, цены - в prices.py. family.py читает семейные профили и объединяет настройки участников.

Стек: Python 3.10+, http.server, sqlite3, SQLite, bcrypt 5.0.0, HTML5, CSS3, JavaScript ES2020, HTTP/JSON.

3 СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ И ИНТЕРФЕЙС

3.1 Варианты использования

Актор - взрослый пользователь. Он регистрируется или входит, задаёт параметры, получает меню, открывает рецепты, выбирает замену, учитывает запасы и сравнивает закупку. Получение списка входит в состав сценария меню, а расчёт целых упаковок - в сравнение корзин. Источник учебных цен является внутренним компонентом, поэтому на диаграмме он не изображён как реальный внешний магазин.

См. рисунок 2 - Диаграмма вариантов использования (приложение А).

3.2 Основной сценарий

После регистрации пользователь проходит обязательный профиль: параметры, активность и цель, необязательные ограничения и сводку. Сохранение открывает дашборд. Отдельный экран плана принимает период 7-31 день, 3-5 приёмов пищи, бюджет периода, время и предпочтения. Параметры человека берутся из сохранённого профиля. Для каждого приёма выбирается допустимый рецепт, затем размер порции масштабируется под долю суточной энергии. Новый план записывается транзакцией только после успешного подбора всех позиций.

Из сохранённого плана формируется единый список, вычитаются домашние запасы, количества округляются до целых товарных упаковок. Полные корзины сортируются по сумме покупки. Неполная корзина показывается отдельно и не рекомендуется как самая дешёвая.

См. рисунок 3 - Основной сценарий (приложение А).

3.3 Альтернативный сценарий

Пользователь открывает блюдо и запрашивает альтернативы. Сервер учитывает тип приёма пищи, аллергены, исключённые продукты, время и повторы в выбранной календарной неделе. Подтверждённая замена меняет только одну позицию. Размер порции сохраняет энергетическую долю

заменяемого приёма; суточные значения, список и корзины пересчитываются. При отмене запрос изменения не отправляется.

В результате меню сгруппировано по дням; у блюда есть калории, оценка цены ингредиентов и кнопка замены. Отдельная корзина сравнивается с бюджетом, превышение выделяется красным. Кандидаты проверяются по калорийности одинаковой базовой порции: допустимо отклонение до 15%. Затем размер выбранной порции уточняется под энергетику приёма пищи.

Для каждого кандидата без записи в базу пересчитывается весь список и лучшая полная корзина. Разница цены показывается до выбора. «Выбрать» применяет замену, а «Сохранить план» фиксирует статус saved. При новой замене статус снова draft. Номер версии защищает от устаревшего окна выбора, конфликт возвращает 409. Полная логика: docs/lab2/replacement_logic.md; исходники: replacements.py и result.js.

Каталог может не дать три допустимые альтернативы, требуемые FR-07: в таком случае прототип честно показывает доступное количество или отсутствие вариантов. Условие «не менее трёх» требует расширения каталога и не объявляется выполненным для любого ввода.

См. рисунок 4 - Альтернативный сценарий замены блюда (приложение А).

3.4 Исключительный сценарий

Кнопка проверки сбоя моделирует отказ источника цен. Адаптер выбрасывает PriceSourceUnavailable, API возвращает 503 с кодом PRICE_SOURCE_UNAVAILABLE. Данные предложений не изменяются. Интерфейс оставляет прежние суммы и показывает понятное сообщение. Это воспроизводимый учебный сценарий, а не сообщение о реальном отказе сети магазина.

Ошибки ввода возвращаются с кодом 422 и словарём полей. Если при заданных исключениях не хватает рецептов, прежний план сохраняется. Приложение не снимает ограничения, не заполняет неизвестные цены нулями и не заменяет рецепт без подтверждения.

См. рисунок 5 - Сценарий ошибки источника цен (приложение А).

3.5 Предварительные экранные формы

До программирования агент подготовил схему пяти экранов. Фазлыев А. А. подтвердил их состав в чате 25.10.2026. Схемы и первый обзор сохранены коммитом 6249f18 до реализации.

См. рисунок 6 - Предварительные экранные формы (приложение А).

Настройки, меню, рецепт с заменой, список и корзины образуют основной сценарий. Вход и регистрация - вспомогательная форма перед ним. На малой ширине навигация и карточки перестраиваются.

3.6 Обновление потока от 26.10.2026

Пользователь явно задал обязательный онбординг из трёх шагов и сводки, затем дашборд и отдельный план. Профиль сохраняется в SQLite, а незавершённый ввод и номер шага восстанавливаются из браузерного черновика по аккаунту. После сохранения профиль становится доступен на дашборде. Прямой переход к генерации без профиля блокируется в UI и API.

Профиль и настройки плана разделены, чтобы новый период не перезаписывал параметры человека. Шкалы ограничивают диапазон и шаг, рядом показано текущее значение. Числовые поля как единственный способ ввода, одна длинная форма и пропускаемый профиль отклонены: они не соответствуют постановке этой итерации. Подробнее: docs/lab2/entry_flow.md и ADR-006.

См. рисунок 8 - Поток регистрации, профиля, дашборда и плана (приложение А).

См. рисунок 9 - Четыре экрана онбординга (приложение А).

3.7 Дополнительные диаграммы активности

Регистрация и заполнение профиля показаны отдельно от настройки периода. После успешной регистрации сервер создаёт пользователя и незавершённый профиль. Интерфейс последовательно показывает три шага и сводку. Переход назад не удаляет ввод; черновик хранится в localStorage в

пределах аккаунта. Ошибка проверки оставляет пользователя в форме. Только успешный PUT /api/profile открывает дашборд.

См. рисунок 11 - Регистрация и заполнение профиля (приложение А).

При генерации API сначала проверяет сессию и завершённый профиль, затем объединяет его с разрешёнными параметрами периода. Аллергии из профиля нельзя снять полями запроса. После полного подбора сервер одной транзакцией создаёт план и позиции, затем формирует снимок со списком и корзинами. Недостаток рецептов не разрушает прежний план. План может превышать бюджет: это состояние результата, а не ошибка записи.

См. рисунок 12 - Генерация меню (приложение А).

4 МОДЕЛЬ ДАННЫХ

4.1 Физическая схема SQLite

Схема хранения приведена в prototype/schema.sql. Для каждого соединения включаются внешние ключи. Идентификаторы предметных сущностей - UUID или стабильные идентификаторы каталога. Деньги хранятся целым числом копеек. Даты предложений и создания - строки ISO 8601 с часовым поясом; начало меню - календарная дата.

См. рисунок 7 - ER-модель прототипа (приложение А).

Таблица 2

Сущность	Первичный ключ	Связи и ограничения
users	id	email уникален; одна учётная запись имеет множество сессий и планов
sessions	token_hash	user_id → users; срок действия сессии
login_attempts	email, attempted_at	Журнал включает и неизвестные email, поэтому FK к users отсутствует
profiles	user_id	PK одновременно FK к users; основной профиль, дата рождения и вычисляемый возраст
family_profiles	id	user_id FK к users; отдельные параметры члена семьи в settings_json и created_at
plans	id	user_id → users; снимок настроек и даты периода
plan_items	id	plan_id → plans, recipe_id → recipes; уникальна тройка plan_id, day, slot
recipes	id	Ингредиенты и шаги - JSON, состав хранится на базовую порцию
ingredients	id	Название, категория, единица, КБЖУ, метки аллергенов
stores	id	Справочник пяти сетей
offers	id	store_id → stores, ingredient_id → ingredients; фасовка, цена, наличие, дата, источник, город
pantry	user_id, ingredient_id	FK к users и ingredients; домашнее количество и отметка покупки

В рецептах ingredients_json содержит ссылки на идентификаторы ингредиентов. На ER-схеме эта связь показана пунктиром, поскольку SQL не обеспечивает для неё внешний ключ. Каталог фиксирован и загружается из согласованного набора catalog.py. Перед вводом редактируемых рецептов потребуется нормализованная таблица recipe_ingredients и миграция данных. Это известный компромисс прототипа, а не заявленная реляционная целостность JSON.

У pantry отметка покупки пока относится к продукту пользователя, а не к отдельной закупке: она переносится на новый план. Это удобно для демонстрации сохранения, но перед рабочим выпуском нужна сущность shopping_list_item с отметкой в пределах конкретной закупки.

4.2 Предметные сущности и связи

Предметная ER-модель содержит шесть требуемых сущностей. Их имена сопоставлены с существующими таблицами, чтобы логическая схема не создавала впечатления, что в базе есть новые таблицы User, Meal или Dish.

Таблица 3

Предметная сущность	Таблица SQLite	Основные атрибуты
Пользователь User	users	id TEXT PK; email TEXT UNIQUE; password_hash TEXT; created_at TEXT
Профиль Profile	profiles	user_id TEXT PK и FK; settings_json TEXT: имя, пол, возраст, вес, рост, активность, цель, ограничения, бюджет, completed
План Plan	plans	id TEXT PK; user_id TEXT FK; start_date TEXT; days INTEGER; settings_json TEXT; created_at TEXT; status TEXT; revision INTEGER; saved_at TEXT nullable
Приём пищи Meal	plan_items	id TEXT PK; plan_id TEXT FK; day INTEGER; slot INTEGER; recipe_id TEXT FK; scale REAL

Предметная сущность	Таблица SQLite	Основные атрибуты
Блюдо Dish	recipes	id TEXT PK; name TEXT; meal TEXT; minutes INTEGER; ingredients_json TEXT; steps_json TEXT
Ингредиент Ingredient	ingredients	id TEXT PK; name TEXT; category TEXT; unit TEXT; kcal, protein, fat, carbs REAL; allergens_json TEXT

См. рисунок 10 - Предметная ER-модель и соответствие SQLite (приложение А).

User связан с Profile как 1 к 0..1 по ограничениям БД. Регистрация сразу создаёт Profile с completed=false, поэтому прикладной поток имеет отношение 1:1, хотя SQL допускает отсутствие профиля.

Связи User с Plan, Plan с Meal и Dish с Meal имеют тип 1:N. Каждый приём принадлежит одному плану и содержит одно блюдо. Нижняя граница на физической схеме равна 0: SQL не требует дочерних строк. Генератор создаёт days × meals позиций одной транзакцией. Индексы day и slot начинаются с нуля; UNIQUE(plan_id, day, slot) запрещает дублирование позиции.

Plan и Dish связаны как M:N через Meal: разные блюда встречаются в разных планах. Таблица plan_items хранит эту связь, день, номер приёма и коэффициент порции; отдельная PlanDish не нужна.

Dish и Ingredient связаны как M:N. Количества на базовую порцию записаны в recipes.ingredients_json, например {"oats": 60, "milk": 180}. Пунктир на схеме обозначает логические ссылки без SQL FK. Для нормализации предложена будущая таблица DishIngredient / recipe_ingredients с PK (recipe_id, ingredient_id), двумя FK и amount. Сейчас её в базе нет.

recipes.meal задаёт тип breakfast/lunch/dinner/snack, а Meal обозначает позицию плана. Калории вычисляются из состава и scale; корзина вычисляется из списка, запасов и предложений. Готовые суммы отдельными столбцами не хранятся. Остальные таблицы показаны на физической схеме, рисунок 7.

5 ПРОГРАММНЫЕ ИНТЕРФЕЙСЫ

5.1 Общие правила

API доступен по пути /api. Формат запросов и ответов - application/json, UTF-8. После входа браузер получает HttpOnly cookie session с SameSite=Strict. На сервере хранится SHA-256 токена, а не сам токен. Наличие авторизации проверяется до чтения пользовательского плана; чужая позиция не возвращается и не заменяется.

Конфликт версии плана возвращает 409 с code: PLAN_CHANGED. Коды ответов: 200 - успешное чтение или изменение; 201 - создан план; 401 - нет сессии или неверные данные входа; 403 - недопустимый Origin; 404 - ресурс не найден; 422 - ошибка полей или невыполнимые ограничения; 429 - превышен лимит попыток входа; 503 - отказ источника цен. Ответ об ошибке имеет поле error, а при ошибке ввода - fields. Тела запросов ограничены 64 КиБ. Пароли и тела запросов не записываются в журнал.

5.2 HTTP контракты

Таблица 4

Метод и маршрут	Вход	Результат
GET /api/session	Cookie, если есть	authenticated и email
POST /api/auth/register	email, password	Создание учётной записи, cookie; 422 при занятом email
POST /api/auth/login	email, password	Cookie; 401 при неверном пароле, 429 после пяти ошибок
POST /api/auth/demo	Пустой объект	Отдельная демонстрационная учётная запись без пароля
POST /api/auth/logout	Пустой объект	Удаление сессии и cookie
GET /api/state	Сессия	Профиль, каталог, текущий план, список и корзины
PUT /api/profile	Параметры человека, дата рождения, активность, цель, ограничения	Сохранённый профиль, вычисляемый возраст и состояние приложения
PATCH /api/profile	Ровно одно изменяемое поле	Обновлённый снимок; остальные поля берутся из БД
POST /api/family	Параметры члена семьи, включая birth_date	201; семейные профили текущего пользователя

Метод и маршрут	Вход	Результат
POST /api/plan	Период, настройки и member_ids	201; нормы участников, общий план, индивидуальные порции и корзины
POST /api/plan/save	plan_id, expected_revision	Статус saved и дата; 409 при устаревшей версии
GET /api/alternatives? item_id=…	ID позиции своего плана	Массив допустимых рецептов
POST /api/replace	item_id, recipe_id, expected_revision	Замена одной позиции, корзина и новая версия; 409 при конфликте
PUT /api/pantry	ingredient_id, amount, purchased	Сохранённый запас и пересчитанный снимок
POST /api/prices/refresh	simulate_failure: boolean	Новый снимок или 503; прежние цены при отказе сохранены

Пример запроса для готового профиля:

```
{"days":7,"meals":3,
"budget":3500,"minutes":30,"repeat":2,
"start_date":"2026-10-26",
"preferences":["no_fish"]}
```

Пример ошибки валидации:

```
{"error":"Проверьте настройки",
"fields":{"days":"Введите число от 7 до 31"}}
```

Пример строки расчёта предложения: ingredient_id = rice, needed = 600 г, pack_size = 500 г, price_kopecks = 10000. Результат: packs = 2, cost_kopecks = 20000, leftover = 400 г. Остаток считается относительно количества, которое нужно купить после вычета запасов.

5.3 JSON контракты ключевых операций

Ниже описаны четыре действующих маршрута из prototype/server.py. Это описание текущего прототипа, без изменения серверного кода. Запросы и ответы передаются как JSON в UTF-8. Для всех четырёх маршрутов требуется cookie session, полученная после регистрации или входа. Пользователь определяется по сессии, поэтому user_id не принимается из браузера.

Примеры ответов сокращены до полей, используемых в данном сценарии. Это проекции полных JSON-ответов, а не отдельные форматы API. Идентификаторы и числовые результаты условны; они поясняют структуру и

не являются результатами нового запуска. Полный снимок содержит также каталог recipes, ingredients, allergens, профиль и признаки состояния. Цена в запросе budget указывается в рублях, поля с суффиксом _kopecks в ответе содержат целые копейки. Поле recommended содержит ID магазина, а не объект корзины.

5.3.1 Сохранение профиля

Метод: PUT. URL: /api/profile. Фронтенд вызывает его на сводке онбординга. Метод PUT сохранён по фактической реализации, хотя генерация и замена используют POST.

JSON-запрос:

```
{
"name": "Азамат", "sex": "male", "age": 22,
"weight": 85, "height": 180, "activity": 1.375,
"goal": "lose", "allergens": ["арахис"],
"preferences": ["no_fish"], "other_allergies": "",
"budget_week": 3500
}
```

HTTP 200. JSON-ответ, проекция снимка без существующего плана:

```
{
"profile": {
"name": "Азамат", "sex": "male", "age": 22,
"weight": 85.0, "height": 180, "activity": 1.375,
"goal": "lose", "allergens": ["арахис"],
"preferences": ["no_fish"], "other_allergies": "",
"budget_week": 3500, "completed": true
},
"profile_complete": true, "plan": null, "test_data": true
}
```

Сервер проверяет обязательные поля, диапазоны, шаг шкал и допустимые значения списков. Повторный PUT обновляет единственную строку profiles данного пользователя. При изменении параметров, влияющих на расчёт, старый план не удаляется, но скрывается из актуального снимка с previous_plan_requires_update: true. Нужна повторная генерация. Ошибка полей возвращает 422, отсутствие сессии возвращает 401.

5.3.2 Генерация меню

Метод: POST. URL: /api/plan. Фронтенд передаёт параметры периода; параметры человека и аллергии сервер берёт из сохранённого профиля.

JSON-запрос:

```
{
"start_date": "2026-10-26", "days": 7, "meals": 3,
"budget": 3500, "minutes": 30, "repeat": 2,
"preferences": ["no_fish"]
}
```

HTTP 201. JSON-ответ, проекция; items и baskets сокращены до одной записи:

```
{
"profile_complete": true,
"plan": {
"id": "plan_example", "start_date": "2026-10-26",
"days": 7, "status": "draft", "revision": 0,
"saved_at": null,
"settings": {"days": 7, "meals": 3, "budget": 3500},
"items": [{
"id": "meal_example", "plan_id": "plan_example",
"day": 0, "slot": 0, "recipe_id": "r01", "scale": 1.2,
"nutrition": {"kcal": 600.0},
"estimated_cost_kopecks": 8500
}]
},
"recommended": "lenta",
"baskets": [{"id": "lenta", "complete": true,
"total_kopecks": 320000, "over_budget_kopecks": 0}],
"test_data": true
}
```

days допускает 7-31, meals допускает 3-5, minutes допускает 5-180, repeat допускает 1-7; бюджет относится ко всему периоду. Индексы day и slot начинаются с нуля. Название блюда фронтенд получает из recipes по recipe_id. nutrition также содержит protein, fat, carbs. Полный settings включает цель, ограничения, target и отпечаток профиля. План и все позиции записываются одной транзакцией после успешного подбора. Если профиль не завершён, возраст менее 18 лет или меню невозможно, возвращается 422; предыдущий план сохраняется. Превышение бюджета само по себе не отменяет генерацию, а отображается в результате.

5.3.3 Получение альтернатив

Метод: GET. URL: /api/alternatives?item_id=meal_example. JSON-тело запроса отсутствует; item_id передаётся в строке запроса. Это ID конкретного приёма пищи, не ID рецепта.

HTTP 200. JSON-ответ, проекция:

```
{
"plan_id": "plan_example", "plan_revision": 0,
"baseline_cart_kopecks": 320000, "baseline_store": "lenta",
"alternatives": [{
"id": "r02", "name": "Овсянка с бананом",
"planned_nutrition": {"kcal": 600.0},
"scale": 1.15, "cart_total_kopecks": 315000,
"cart_delta_kopecks": -5000, "recommended_store": "lenta"
}]
}
```

Состав проверяется по ограничениям, после чего для каждого кандидата пересчитывается вся закупка с запасами и упаковками. GET ничего не записывает. При отсутствии вариантов alternatives равен []. Если полной корзины нет, соответствующая сумма и разница равны null, а не нулю. Чужая или отсутствующая позиция даёт 404. Изменённый профиль требует новой генерации и даёт 422.

5.3.4 Замена блюда

Метод: POST. URL: /api/replace. Нужны оба идентификатора: item_id указывает место в плане, recipe_id указывает выбранное блюдо. expected_revision берётся из plan_revision ответа альтернатив. Пользователь и план повторно проверяются сервером.

JSON-запрос:

```
{
"item_id": "meal_example", "recipe_id": "r02",
"expected_revision": 0
}
```

HTTP 200. JSON-ответ, проекция обновлённого снимка:

```
{
"plan": {
"id": "plan_example", "status": "draft", "revision": 1,
"saved_at": null,
"items": [{"id": "meal_example", "recipe_id": "r02",
"scale": 1.15, "nutrition": {"kcal": 600.0},
"estimated_cost_kopecks": 8000}]
},
"recommended": "lenta",
"baskets": [{"id": "lenta", "complete": true,
"total_kopecks": 315000, "over_budget_kopecks": 0}]
}
```

Новое блюдо возвращается в plan.items, новая сумма находится в baskets у recommended. API не возвращает отдельные new_dish или new_total: таких

полей в реализации нет. Смена рецепта меняет только указанную позицию и увеличивает revision. Сохранённый план становится черновиком. Сервер заново фильтрует кандидата внутри транзакции, не доверяя открытому ранее окну браузера.

Пример HTTP 409 при устаревшей версии:

```
{"error": "План изменился. Запросите альтернативы заново.",
"code": "PLAN_CHANGED"}
```

Недопустимая замена возвращает 422, неизвестная или чужая позиция возвращает 404. При ошибке изменения не фиксируются. Для сохранения результата используется существующий POST /api/plan/save с plan_id и expected_revision; он возвращает снимок со status: saved и saved_at.

5.4 Внутренние контракты

```
validate_settings(data: dict) -> dict
generate_plan(settings: dict, recipes: dict, ingredients: dict) -> list[dict]
nutrition(recipe: dict, ingredients: dict, scale: float = 1) -> dict
shopping(items: list, recipes: dict, ingredients: dict, pantry: dict) -> list[dict]
calculate_baskets(lines: list, stores: list, offers: list, budget: float) -> dict
DemoPriceProvider.read(db) -> list[dict]
DemoPriceProvider.refresh(db, simulate_failure: bool = False) -> None
```

Это контракты документации; функции Python не везде содержат аннотации типов в исходнике. validate_settings нормализует значения и возвращает target с калориями и БЖУ. generate_plan возвращает позиции day, slot, recipe_id, scale или выбрасывает DomainError. shopping возвращает required, at_home, needed, purchased и единицу. calculate_baskets возвращает baskets, recommended и saving_kopecks. Адаптер цен при отказе выбрасывает PriceSourceUnavailable до изменения базы.

6 АРХИТЕКТУРНЫЕ РЕШЕНИЯ

Приняты десять ADR. Новые ADR-007–010 уточняют SQLite, Python, серверные расчёты и фильтрацию аллергенов. ADR-006 о разделении профиля и плана сохранён. Полные тексты приведены далее и в docs/lab2/adr/.

ADR-001: Модульный монолит с браузерным интерфейсом

Статус

Принято

Контекст

FR-05–15 связывают меню, список и стоимость. Изменение блюда должно согласованно пересчитывать производные данные. FR-01 требует входа и сохранения данных пользователя, NFR-05 - браузерного интерфейса на нескольких ширинах. Команде нужен запускаемый срез без отдельной инфраструктуры для каждого компонента.

Рассмотренные альтернативы

1. Микросервисы: дают независимое развёртывание, но для первого среза добавляют сетевые отказы, отдельные конфигурации и сложность транзакций.

2. Полностью клиентское приложение: проще запуск, однако хранение только в браузере не обеспечивает данные учётной записи при входе из другой сессии на сервере.

3. Desktop-приложение: удобно локально, но требует упаковки под платформы и расходится с браузерным направлением исходной презентации.

Принятое решение

Один Python-процесс отдаёт статический HTML/CSS/JavaScript и HTTP API. Вычисления вынесены в domain.py, источник цен - в prices.py; транспорт и SQL пока находятся в server.py. В браузере нет второго независимого механизма расчёта корзины: он отображает согласованный снимок сервера.

Последствия

Упрощаются запуск, отладка и транзакционная запись плана. Усложняется дальнейшее выделение авторизации и хранения из HTTP-обработчика. Риск - рост server.py при добавлении функций; границы уже заданы контрактами. Стандартный HTTP-сервер годится для локальной демонстрации, но требует замены или production-обвязки при публикации.

Роль агента

Агент предложил серверное хранение после проверки FR-01 сданной ЛР1 и выбрал небольшой модульный монолит. Пользователь подтвердил набор экранов до программирования и уточнил аудиторию. Выбор конкретной серверной реализации выполнен агентом в рамках порученной разработки; отдельное сравнение технологий пользователем не приписывается.

ADR-002: Серверная SQLite и снимок настроек плана

Статус

Принято

Контекст

FR-01 требует сохранить данные учётной записи; FR-10 - запасы после повторного входа; FR-05 и FR-07 - целостное меню и замену одной позиции. Нельзя записать половину нового плана и потерять предыдущий при нехватке рецептов.

Рассмотренные альтернативы

1. PostgreSQL: подходит для многопользовательского развёртывания, но отдельный сервер и настройка избыточны для локальной защиты.

2. JSON-файл: прозрачен, но потребовал бы ручной координации одновременных записей, уникальности и связей.

3. localStorage: удобен для черновиков UI, но привязан к конкретному браузерному профилю и не выполняет серверную часть FR-01.

Принятое решение

Использовать SQLite через sqlite3. Включать foreign_keys на каждом соединении, передавать значения через SQL-параметры, закрывать соединения явно. Записывать план и позиции одной транзакцией; сначала полностью рассчитывать допустимое меню в памяти. Сохранять settings_json в плане, чтобы последующие изменения формы не меняли трактовку уже созданного периода. Состав фиксированного рецепта пока хранится в JSON.

Последствия

Упрощаются резервное копирование и запуск. Усложняются конкурентная запись при росте нагрузки и запросы по JSON-составу. Риск нарушения ссылок внутри JSON ограничен фиксированным каталогом, но не устранён SQL. Перед редактированием каталога нужны нормализация recipe_ingredients и миграции; дальнейшая база не обещана неизменной.

Роль агента

Агент сформировал ER по фактической схеме, отметил ограничение JSON и добавил API-проверку сохранения предыдущего плана. Тест на Windows обнаружил, что контекст SQLite сам по себе не закрывал соединение; агент ввёл явное закрытие. Пользователь не заявлял эти проверки как выполненные им лично.

ADR-003: Детерминированный расчёт меню и целых упаковок

Статус

Принято

Контекст

FR-02 задаёт пример нормы калорий; FR-05 требует ограничений меню; FR-07 - пересчёта после замены; FR-09–13 - списка и целых упаковок; FR-15 - проверки бюджета. В FR-17 уже указано, что числа для анализа должен считать код.

Рассмотренные альтернативы

1. Расчёт языковой моделью: гибкие объяснения, но недетерминированная арифметика и дополнительные расходы.

2. Полный математический оптимизатор: может учитывать много ограничений одновременно, но потребовал бы отдельной постановки задачи и усложнил первый срез.

3. Ручное меню без автоматического подбора: надёжно для демонстрации карточек, но не показывает основной сценарий FR-05.

Принятое решение

Использовать чистые Python-функции. Кандидаты фильтруются по составу, времени и повторяемости, затем порция масштабируется под энергетическую долю приёма. Список суммирует канонические количества, вычитает запасы. Для каждого предложения количество упаковок равно ceil(потребность / фасовка), стоимость считается целыми копейками. Рекомендуются только полные корзины; экономия сравнивается со следующей полной корзиной.

Последствия

Упрощаются воспроизводимость и проверка на примерах. Подбор не гарантирует оптимум бюджета или целевых БЖУ, а масштабирование может дать неудобные кулинарные порции. Эти ограничения видны в прототипе и требуют развития. Нельзя представлять приближённый учебный каталог как основу медицинских рекомендаций.

Роль агента

Агент написал расчёты и граничные тесты, включая неполную дешёвую корзину. Проверка примера ЛР1 выявила расхождение округления 234,5 г; агент согласовал правило half-up между сервером и интерфейсом. Пользователь уточнил фокус на похудении; это отражено в основном сообщении продукта, а не в вымышленных гарантиях результата.

ADR-004: Заменяемый адаптер демонстрационных цен

Статус

Принято

Контекст

FR-11 требует цены пяти сетей с датой и наличием; FR-12–13 используют фасовки; NFR-03 требует актуальности. Реальный доступ к данным сетей пока не подтверждён. Для ЛР2 нужен воспроизводимый сценарий интерфейса и обработки отказа источника.

Рассмотренные альтернативы

1. Сразу парсить пять сайтов: зависит от региона, защиты и формата страниц; доступ и условия сбора пока не проверены.

2. Убрать сравнение до появления API: исключило бы важный экран и ключевую ценность из демонстрации.

3. Захардкодить итоговые суммы в UI: быстро, но не проверяет упаковки, замену блюд и взаимосвязь меню со стоимостью.

Принятое решение

Создать DemoPriceProvider с методами read и refresh. Исходные предложения содержат сеть, ингредиент, размер упаковки, цену в копейках, наличие, дату, источник и город. Суммы вычисляются на этих данных. В интерфейсе постоянно указано, что цены тестовые. Моделируемый отказ выбрасывается до записи и возвращает 503; прошлые предложения остаются доступными. Будущий внешний адаптер должен сохранить эти контракты.

Последствия

Упрощаются повторяемая защита и тестирование ошибок. Нельзя делать вывод об экономии в настоящих магазинах или считать выполненным суточное обновление NFR-03. Риск - принять учебные данные за реальные; он снижен явной маркировкой и документированием. Доставка в первом сравнении не учитывается: сценарий обозначен как самовывоз.

Роль агента

Агент отделил интерфейс источника от расчётов и добавил тест неизменности корзин при 503. Решение опирается на ранее обсуждённый риск доступа к ценам. Пять реальных сетей не выдаются за интегрированные; агент не приписывает пользователю договорённости с магазинами.

ADR-005: Хеширование паролей и серверные сессии

Статус

Принято

Контекст

FR-01 предусматривает email и пароль. NFR-01 требует bcrypt cost 12, HTTPS и блокировку после пяти неверных попыток за 15 минут. Доступ к позиции плана должен проверять владельца, а демонстрация - не требовать публикации настоящих учётных данных.

Рассмотренные альтернативы

1. Пароль или обратимо зашифрованный пароль в базе: не соответствует NFR-01 и создаёт ненужный риск раскрытия.

2. JWT в localStorage: удобен для распределённой системы, но сложнее отзыв сессии, а токен доступен JavaScript.

3. Внешний OAuth-провайдер: убирает локальный пароль, но требует внешней настройки и расходится с конкретным требованием email/пароль первой версии.

Принятое решение

Хешировать пароль bcrypt с rounds=12. Создавать случайную cookie-сессию; сохранять только SHA-256 токена, срок действия и владельца. Cookie имеет HttpOnly, SameSite=Strict и Path=/. Проверять Origin изменяющих запросов и принадлежность данных. После пяти ошибок для email за последние 15 минут отклонять дальнейший вход до истечения окна. При выходе удалять серверную сессию. Демонстрационный вход создаёт отдельную учётную запись с отключённым паролем.

Последствия

Упрощается отзыв сессий и ограничивается доступ JavaScript к токену. Возникают расходы bcrypt и необходимость хранения сессий. Локальный HTTP на loopback не выполняет HTTPS-часть NFR-01; Secure-cookie, TLS и production-развёртывание должны быть добавлены перед доступом через сеть. Демонстрационные аккаунты требуют будущей очистки по сроку жизни.

Роль агента

Агент сверился с конкретным NFR-01 сданного отчёта и выбрал bcrypt вместо замены алгоритма без объяснения. Добавил проверку префикса хеша cost 12, входа второй сессии, изоляции другого пользователя и блокировки попыток. Выбор и его ограничения сформулированы агентом; отсутствующие production-проверки не приписаны студентам.

ADR-006: Обязательный профиль отдельно от плана

Статус

Принято

Контекст

FR-01 предусматривает регистрацию, FR-02 и FR-03 задают параметры человека и исключения, FR-05 и FR-15 относятся к плану на выбранный период. В задании пользователя от 26.10.2026 после регистрации обязателен профиль из трёх шагов и сводки, затем дашборд и отдельная настройка плана.

Рассмотренные альтернативы

1. Одна форма: меньше экранов, но смешивает устойчивые данные с периодом и не соответствует заданному потоку.

2. Пропускаемый профиль: быстрее открывает приложение, но допускает расчёт без обязательных исходных данных.

3. Только localStorage: не требует серверного маршрута, но готовый профиль оказывается привязан к одному браузеру и расходится с существующей авторизацией.

Принятое решение

Хранить профиль в profiles отдельно от снимка settings_json конкретного плана. Ввести PUT /api/profile и серверную проверку profile_complete перед генерацией. Черновик шагов хранить в localStorage по аккаунту; после успешного сохранения удалять его и переходить на дашборд. Для ограниченных чисел применять заданные пользователем шкалы, подтверждая значения серверной валидацией. В плане сохранять отпечаток значимых параметров профиля, чтобы не выдавать старый рацион за актуальный после изменения исключений.

Последствия

Повторный план не требует заново заполнять профиль. Возврат и перезагрузка не стирают черновик. Усложняется синхронизация черновика и готового профиля; теперь есть два состояния данных. Старый план после изменения параметров нужно пересоставить, хотя запись сохраняется. Дополнительный API требует проверки входных значений и принадлежности аккаунту. Автоматический расчёт для возраста 14-17 лет в этой итерации не добавляется.

Роль агента

Пользователь сформулировал явную задачу с четырьмя состояниями онбординга и дашбордом. Агент разделил модели, реализовал формы и серверную защиту, сохранил заданные диапазоны шкал, добавил тесты возврата, перезагрузки и обхода маршрута. Отпечаток плана предложен и реализован агентом как защита от скрытого использования старых ограничений. Числовые поля и одна общая форма рассмотрены при реализации и отклонены по требованиям этого запроса.

ADR-007: SQLite для локального прототипа

Статус

Принято. Уточнение архитектуры действующего прототипа от 26.10.2026.

Контекст

Данные профиля, планов и каталога уже хранятся через sqlite3. FR-01 и FR-10 требуют сохранения между сеансами, FR-05 и FR-07 требуют целостных изменений. ADR-002 зафиксировал способ хранения; этот ADR уточняет выбор СУБД для текущего масштаба.

Проблема

Нужна база, которую можно запустить на защите без отдельного сервиса, сохранив транзакции, уникальность и внешние ключи.

Рассмотренные альтернативы

PostgreSQL требует отдельного сервера, конфигурации доступа и доставки окружения. JSON-файл потребовал бы ручной реализации связей и согласованной записи. localStorage подходит для черновика формы, но не для общей серверной базы.

Принятое решение

Оставить SQLite. Соединение создаётся на запрос, foreign_keys включается явно. Новый план и его позиции записываются вместе; замена использует BEGIN IMMEDIATE и номер revision. SQL-значения передаются параметрами. Это уточнение ADR-002, а не смена хранилища.

Последствия

Упрощаются установка и резервное копирование. Записи конкурируют за блокировку файла; многопользовательская нагрузка в этой итерации не измерялась. При переходе к публичному сервису потребуется оценить PostgreSQL и подготовить миграции. JSON-состав рецептов пока не имеет SQL FK.

Роль агента

Агент сопоставил решение с schema.sql и обработчиками server.py. В этой документальной итерации код и база не изменялись; новые испытания производительности не проводились.

ADR-008: Python для серверной логики

Статус

Принято. Уточнение архитектуры действующего прототипа от 26.10.2026.

Контекст

FR-02 требует сохранения параметров и расчёта нормы, FR-05 - составления меню, FR-07 - замены блюда с пересчётом. NFR-01 задаёт bcrypt cost 12 и ограничение попыток входа; NFR-02 задаёт p95 генерации до 5 с на условиях ЛР1. Python объединяет эти правила на сервере: domain.py выполняет расчёты, profiles.py проверяет профиль, replacements.py обрабатывает замены. Выбор языка сам по себе не подтверждает выполнение NFR-02: соответствующие нагрузочные проверки не проводились. HTTPS из NFR-01 пока не реализован.

Проблема

Нужно сохранить понятный и воспроизводимый прототип без сборки фронтенда и без повторной реализации готовых расчётов на другом языке.

Рассмотренные альтернативы

Node.js позволил бы использовать JavaScript на обеих сторонах, но потребовал бы переноса расчётов и слоя БД. Python с FastAPI дал бы декларативные контракты, но добавил бы зависимости и настройку приложения. Переписывание на Java или C# не решает задачу текущей итерации.

Принятое решение

Продолжать использовать Python 3.10+ и стандартный ThreadingHTTPServer для локальной демонстрации. Оставить один серверный процесс и модули по ответственности. Внешняя зависимость приложения нужна для bcrypt; инструменты построения отчёта не входят в рабочий стек.

Последствия

Сохраняется простой запуск и единое место вычислений. Валидация маршрутов написана вручную, интерактивная спецификация OpenAPI отсутствует. http.server не является выбранным сервером публичного продукта. Перед публикацией нужны сервер приложений, HTTPS и пересмотр эксплуатации.

Роль агента

Агент описал существующее разбиение модулей и альтернативы без изменения стека. Заявление о сравнительной скорости языков не делается, соответствующих измерений нет.

ADR-009: Расчёт калорий и корзин на сервере

Статус

Принято. Уточнение архитектуры действующего прототипа от 26.10.2026.

Контекст

FR-02, FR-05, FR-07 и FR-09–15 связывают нормы, порции, список, упаковки и бюджет. ADR-003 определил детерминированную арифметику. Данные профиля и предложения магазинов находятся на сервере.

Проблема

Расчёт в браузере и на сервере может разойтись из-за округления, устаревших данных или изменённого клиентского запроса. Нельзя принимать из UI готовую стоимость и считать её результатом системы.

Рассмотренные альтернативы

Расчёт только в браузере уменьшает число запросов, но дублирует правила и позволяет подменить итог. Две независимые реализации повышают стоимость сопровождения. Внешняя языковая модель не нужна для арифметики и не используется.

Принятое решение

Считать целевые нормы, порции, допустимые замены, список и полные корзины Python-функциями на сервере. Фронтенд отправляет параметры и идентификаторы, затем отображает снимок. Копейки целочисленные, число упаковок округляется вверх. Прогноз замены применяет те же shopping и calculate_baskets, что итог. Решение уточняет границу компонентов из ADR-003.

Последствия

Сервер остаётся источником расчётного результата, едины правила округления и ограничений. Для замены требуется сетевой запрос; автономная работа пока не предусмотрена. Предварительная оценка перебирает кандидатов и пересчитывает корзину, поэтому рост каталога потребует замеров и возможного кэширования. Оптимальность бюджета не гарантируется.

Роль агента

Агент проверил описание по вызовам snapshot и preview_replacements. В отчёте сохранены ранее полученные результаты, но новые тесты в этой итерации не запускались.

ADR-010: Повторная проверка аллергенов при замене

Статус

Принято. Уточнение архитектуры действующего прототипа от 26.10.2026.

Контекст

FR-03 задаёт исключения, FR-07 требует допустимую замену. Пользователь может открыть окно вариантов, затем изменить профиль или план в другом окне. Каталог содержит метки аллергенов ингредиентов.

Проблема

Фильтра только в интерфейсе недостаточно: клиент может отправить произвольный recipe_id, а открытый список может устареть.

Рассмотренные альтернативы

Клиентская проверка даёт быстрый отклик, но не является границей правил. Проверка только при GET не защищает последующий POST. Снятие ограничений при отсутствии вариантов противоречит потребности пользователя.

Принятое решение

Использовать общий replacement_candidates для GET /api/alternatives и повторно для POST /api/replace. allowed проверяет каждый ингредиент: пересечение его аллергенов с запретами должно быть пустым, исключённые продукты запрещены. Также учитываются предпочтения, время, тип приёма, повторы и диапазон калорий ±15% до подстройки порции. POST проверяет владельца, отпечаток профиля и revision в транзакции. При нарушении возвращается ошибка без записи.

Последствия

Предварительный список и применение подчиняются одним правилам. При отсутствии вариантов ограничения сохраняются. Корректность зависит от полноты меток фиксированного каталога: следы аллергенов и производственные загрязнения не моделируются. Неопознанный текст из «Других аллергий» блокирует генерацию до уточнения, а не игнорируется.

Роль агента

Агент сопоставил решение с allowed, plan_settings и обработчиком замены, отметил зависимость от данных каталога. В этой итерации подготовлены документы и схемы без изменения фильтра и запуска тестов.

7 ПРОВЕРКА ПРОТОТИПА И ПОКРЫТИЕ ТРЕБОВАНИЙ

7.1 Функциональная проверка

Автоматические тесты проверяют не только наличие интерфейса, но и граничные условия расчётов: 31 день при пяти приёмах пищи, повторы в календарной неделе, запрет аллергенов, отсутствие допустимого меню, округление упаковок, неполные корзины и экономию относительно следующей полной корзины. API-тесты используют временную SQLite, проверяют хеш bcrypt с cost 12, вход другой сессии в ту же учётную запись, изоляцию чужих планов, блокировку попыток, сохранение плана после ошибки и отказ источника цен.

Браузерный сценарий проверяет демонстрационный вход, генерацию, замену, запасы, пять сетей, сохранение после перезагрузки и ошибку источника. Фактические результаты записываются в docs/lab2/test_results.md и машинный JSON. Наличие теста не подменяет проведённую проверку; результаты в документе приводятся после запуска.

7.2 Уточнение NFR-05

Целевая проверяемая матрица: Google Chrome 155.0.8059.85 на Windows; Mozilla Firefox 158.0 на Windows; Safari 26.0 на macOS. Основные экраны настроек, меню, списка и корзины должны быть доступны на ширинах 360, 768 и 1920 CSS px без горизонтальной прокрутки страницы. Номер ОС, версия браузера, ширина и результат заносятся в протокол. Это зафиксированные версии для приёмки, а не подвижное обещание поддержки «двух последних версий».

В текущем окружении фактически проверен Google Chrome 155.0.8059.85. Проверки Firefox 158.0 и Safari 26.0 остаются пунктами приёмки; они не объявлены пройденными. Мобильная ширина в настольном браузере проверяет компоновку, но не заменяет испытания на реальном телефоне.

7.3 Матрица покрытия

Таблица 5

Требования ЛР1	Состояние прототипа
FR-01	Регистрация и вход, серверное хранение и повторный вход реализованы. Доступ с другого устройства требует сетевого развёртывания
FR-02	Профиль: возраст 14-90, вес 40-200 с шагом 0,5, рост 140-220. Автоматический расчёт текущего прототипа доступен с 18 лет
FR-03	14 меток аллергенов, исключения и проверка состава реализованы для фиксированного каталога
FR-04	Семейные профили взрослых и общая корзина реализованы; см. раздел 9
FR-05, FR-06	7–31 день, 3–5 приёмов, масштабирование энергии, повторы; при нехватке блюд явная ошибка
FR-07	Подтверждаемая замена и пересчёт; число альтернатив зависит от каталога, гарантия трёх вариантов ещё не обеспечена
FR-08	Карточка, состав, шаги, КБЖУ и количество на приём; самостоятельный выбор числа порций ещё не реализован
FR-09, FR-10	Общий список, категории, запасы и купленные позиции реализованы; каталог использует канонические единицы
FR-11	Модель пяти сетей реализована на тестовых данных; реального получения цен нет
FR-12	Упаковки рассчитываются; сопоставление фиксировано в учебном каталоге. Ручной выбор товарного аналога ещё не реализован
FR-13, FR-15	Полнота, сумма, следующая полная корзина, дата и бюджет работают на тестовом наборе
FR-14, FR-16–19	Два магазина, экономные замены, внешний ИИ, шаблоны и учёт фактических расходов отложены
FR-20, FR-21	Заказ и оплата исключены из текущей версии по Won't
NFR-01	bcrypt и лимит попыток реализованы. HTTPS и production-развёртывание ещё не выполнены
NFR-02, NFR-04	Специальные нагрузочные испытания на условиях ЛР1 не проводились
NFR-03	Даты и сохранение цен при ошибке есть; реальное суточное обновление пяти сетей не реализовано
NFR-05	Зафиксированы версии. Пройденная часть матрицы указана в протоколе

Прототип реализует ключевые сценарии ЛР2; полная спецификация ЛР1 остаётся планом развития.

8 РАБОТА С ИИ АГЕНТОМ

В документальной итерации пользователь задал: «Нам нужно завершить архитектурную часть ЛР2 для отчета», затем перечислил шесть сущностей ER, JSON-контракты, четыре дополнительных ADR и два процесса activity. Агент сверил требования с schema.sql, server.py, profiles.py, domain.py и replacements.py. При сверке уточнены PUT для профиля, два идентификатора при замене и логическая M:N связь состава в JSON. Подготовлены ADR-007–010 и схемы с редактируемыми исходниками. По просьбе пользователя тесты в этой итерации не запускались; протокол приложения содержит результаты предыдущей итерации.

Следующая явная задача пользователя: «Реализуй экран результата после нажатия кнопки "Составить меню"». В постановке заданы дни, калории, цены, бюджет, сохранение и замены с влиянием на всю корзину. Агент реализовал экран, общий фильтр и предварительный пересчёт, затем проверил равенство прогноза и итоговой суммы. Activity-схема обновлена по фактическому процессу. Это отдельная реальная итерация разработки после онбординга.

Следующая итерация началась с явной постановки пользователя: «Спроектируй и реализуй поток входа в Fit Nutrition Supply». Пользователь задал регистрацию, три шага профиля со сводкой, дашборд и отдельный план, точные диапазоны шкал и правила переходов. Агент реализовал поток, сравнил способы хранения, сформировал две редактируемые схемы и добавил проверки. Постановка и результат этой итерации сохранены в docs/lab2/prompts/2026-10-26-entry-flow.md и docs/lab2/entry_flow.md.

Пользователь передал агенту сданные материалы и замечания. До начала кода агент запросил уточнение аудитории, согласование экранов и смысл замечания NFR-05. Пользователь заменил предложенную агентом группу студентов на людей с интересом к похудению, подтвердил экраны и уточнил, что нужны версии браузеров. Эти ответы изменили проект до реализации.

Дословные значимые фрагменты запроса: «надо уже вторую лабу делать - реализовывать начинать продукт»; уточнение ЦА: «ЦА - люди с интересом в похудании»; ответ по экранам: «Да, подходит»; ответ по NFR-05: «указать версии браузеров вроде». Пунктуация первой фразы приведена к читаемому виду без изменения смысла. Полная переписка не подменяется вымышленными промптами.

Агент самостоятельно сформулировал техническую подзадачу: реализовать согласованные экраны, серверную авторизацию и хранение; вынести вычисления в чистые функции; отделить демонстрационный источник цен; сформировать схемы из описания компонентов и согласовать ER с SQL; проверить основной, альтернативный и исключительный сценарии. Это рабочая формулировка агента, она не приписывается студенту как дословный запрос.

В ходе проверки обнаружены и исправлены ошибки: число SQL-параметров при первичной загрузке каталога; расхождение банковского округления Python с примером углеводов из ЛР1; незакрытое соединение SQLite, мешавшее удалению временной базы в Windows; лишние контуры фокуса из-за CSS-селектора; доступное имя кнопок навигации с декоративным символом. После исправлений тесты запущены повторно.

Агент подготовил связанный первый срез, контракты, схемы и тесты. При этом каталог требует проверки, реальные сети ещё не подключены, а покрытие требований неполное. Неисполненные проверки обозначены в протоколе.

9 UX-ДОРАБОТКИ ПОСЛЕ ТЕСТИРОВАНИЯ

По запросу пользователя от 30.10.2026 улучшены навигация, профиль, семейное планирование и пустые экраны. Результат проверен браузерными сценариями на тестовых аккаунтах.

9.1 Общая навигация

Левое меню доступно на дашборде, странице профиля, в настройках плана, результате, меню, списке и корзинах. Активный раздел отмечен визуально и атрибутом aria-current. На малой ширине закреплена кнопка «Меню», открывающая те же разделы. До сохранения первичного профиля расчётные страницы недоступны: обязательные параметры нельзя обойти прямым URL. После завершения профиля переходы не запускают онбординг повторно.

9.2 Просмотр и редактирование профиля

«Мой профиль» открывает карточку со списком параметров. Кнопка «Изменить» у каждого поля показывает отдельный диалог. PATCH /api/profile принимает ровно одно поле, например {"weight": 81.5}; сервер объединяет его с текущей строкой профиля и проверяет результат. Отмена не отправляет изменение. Такой контракт сохраняет остальные значения и не требует повторного прохождения шагов.

В онбординге вводится birth_date, возраст сервер вычисляет по полным годам при чтении и составлении плана. Переданный клиентом age не подменяет вычисление, если указана дата рождения. Старым профилям дата не назначается автоматически: до её ввода используется сохранённый возраст. Возраст профиля ограничен 14-90 годами, автоматический рацион доступен с 18 лет. Изменение расчётных параметров делает прежний план неактуальным, сохраняя его в базе.

9.3 Семейное планирование

В настройках добавлена кнопка «Добавить члена семьи». Можно выбрать ранее сохранённый профиль либо создать новый в диалоге на этой странице. После сохранения участник сразу выбран для текущего плана. Его можно убрать из состава, не удаляя сам профиль. Параметры периода и введённый бюджет при этом сохраняются. Владелец аккаунта включён в план всегда; дополнительно можно выбрать до девяти участников.

POST /api/family сохраняет имя, дату рождения, параметры тела, цель, активность и ограничения. Таблица family_profiles содержит id TEXT PK, user_id TEXT FK на users, settings_json TEXT и created_at TEXT. Связь пользователя с семейными профилями имеет тип 1:N. Доступ к чужому профилю через member_ids отклоняется. На физической ER-схеме показана новая таблица; на предметной схеме Profile по-прежнему обозначает основной профиль аккаунта.

POST /api/plan принимает member_ids, например {"days": 7, "budget": 7000, "member_ids": ["идентификатор"]}. Сервер рассчитывает target для каждого участника. Суточные нормы суммируются для общего подбора и закупки; порция каждого человека пропорциональна его целевой энергии. Используются общие блюда, а не независимые меню. Аллергии, исключения и предпочтения объединяются, поэтому блюдо должно подходить всем. При невозможности подбора ограничения сохраняются и возвращается ошибка.

Состав участников, нормы и отпечатки параметров записываются в plans.settings_json. Ссылки member_ids проверяются приложением, SQL FK внутри JSON отсутствует. Для дальнейшей нормализации может потребоваться plan_participants. Результат содержит общую энергию, персональные нормы и portions для каждого приёма пищи. Корзина суммирует продукты на всех, затем вычитает общие домашние запасы и округляет упаковки. Бюджет относится ко всей закупке и автоматически не умножается.

В учебном сценарии Азамат получил норму 2345 ккал, Анна с целью поддержания - 1917 ккал; общий план составил 4262 ккал в день. Замена

общего блюда сохраняет долю каждого участника и пересчитывает полную семейную корзину. Эти числа получены на демонстрационном каталоге, а не на актуальных ценах магазинов. Автоматический расчёт для несовершеннолетних членов семьи пока не поддерживается.

9.4 Дружелюбные пустые состояния

Без плана разделы результата и меню показывают иконку, текст «У вас пока нет плана» и кнопку «Составить мой первый план». Кнопка открывает настройки. Вместо отсутствующих значений пользователь получает понятное следующее действие. Если прежний план потерял актуальность после изменения профиля, дополнительно объясняется необходимость нового расчёта.

9.5 Проверка доработок

Пройдена 31 автоматическая проверка Python. Новые API-сценарии проверяют индивидуальные порции и суммарные продукты, общие ограничения, изоляцию семейных профилей, запрет повторных ID, сохранность прежнего плана при недопустимом участнике и совпадение прогноза цены с итогом замены. Проверки даты рождения и редактирования одного поля также сохранены.

В Chrome 156.0.8078.79 пройдены сценарии onboarding.cjs, result.cjs, browser.cjs, profile-navigation.cjs и family-ux.cjs. Проверены создание и повторный выбор участника, сохранение состава после перезагрузки, пустые состояния, навигация и мобильная компоновка. На 360/768/1440 px семь основных экранов не имеют горизонтального переполнения. Семейный результат отдельно проверен на 360 и 1440 px. Физический телефон, Firefox и Safari в этом прогоне не использовались.

Скриншоты и JSON результатов: output/family-ux/ и output/profile-navigation/.

ЗАКЛЮЧЕНИЕ

Подготовлены структурная схема, use case, пять диаграмм активности, физическая и предметная ER-модели, JSON-контракты, экраны и десять ADR. Прототип включает регистрацию, обязательный профиль, дашборд, отдельный план, замену блюд, список и пять учебных корзин. Документы и код находятся в одном репозитории. Следующий этап: проверенный каталог и реальный разрешённый источник цен.

ИСТОЧНИКИ

1. Задание ЛР2, предоставленное преподавателем: sources/lab2-assignment.txt.

2. Сданный отчёт ЛР1 и презентация: sources/submitted-lab1/.

3. Python sqlite3: https://docs.python.org/3/library/sqlite3.html.

4. bcrypt 5.0.0: https://pypi.org/project/bcrypt/5.0.0/.

5. Firefox 158.0, примечания к выпуску: https://www.firefox.com/en-US/firefox/158.0/releasenotes/.

6. Safari 26.0, примечания к выпуску: https://developer.apple.com/documentation/safari-release-notes/safari-26-release-notes.

7. Chrome, документация версий: https://developer.chrome.com/release-notes. Версия фактически установленного браузера зафиксирована автоматической проверкой.

Приложение А Схемы

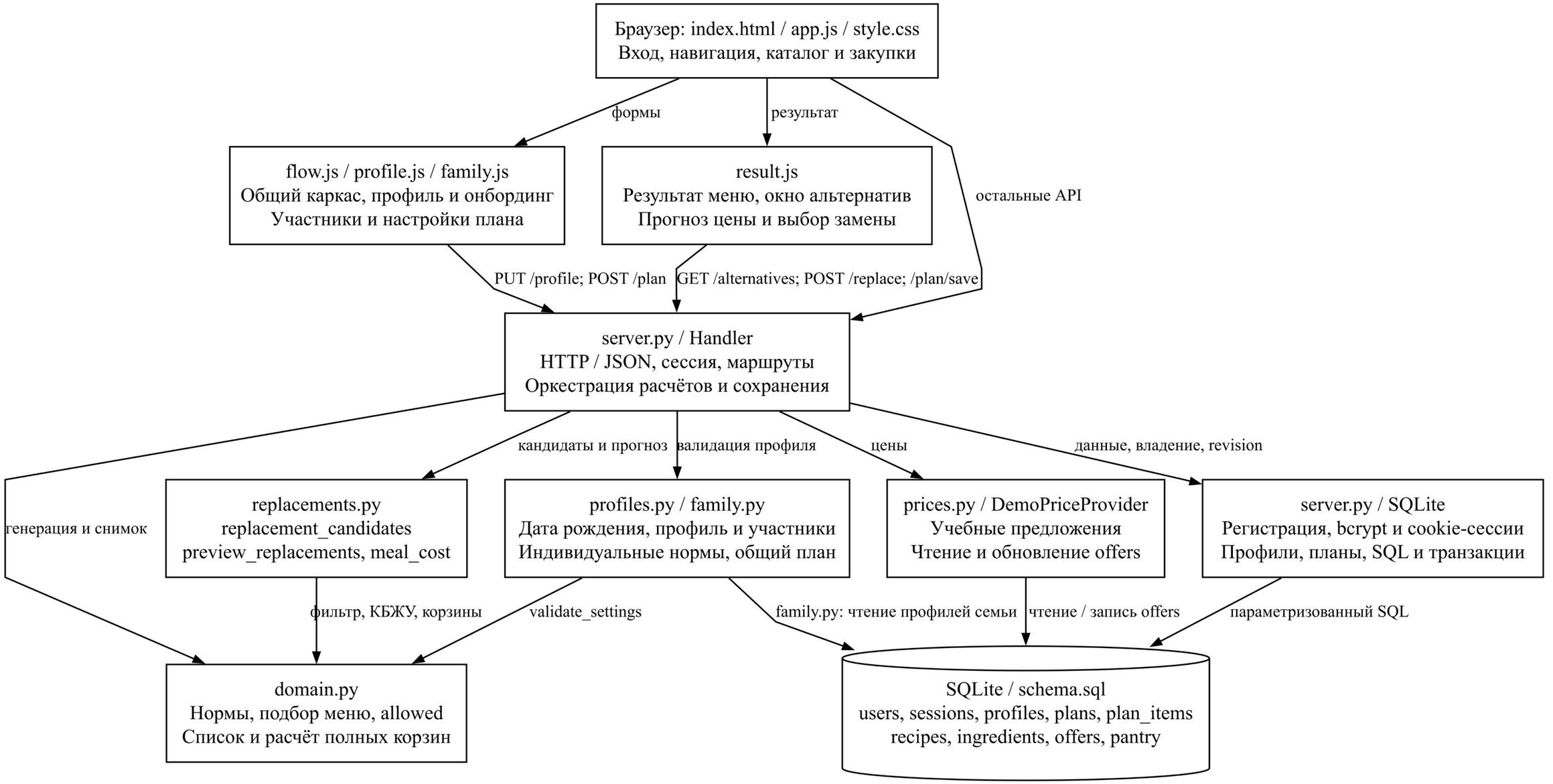


Рисунок 1 - Структурная схема системы

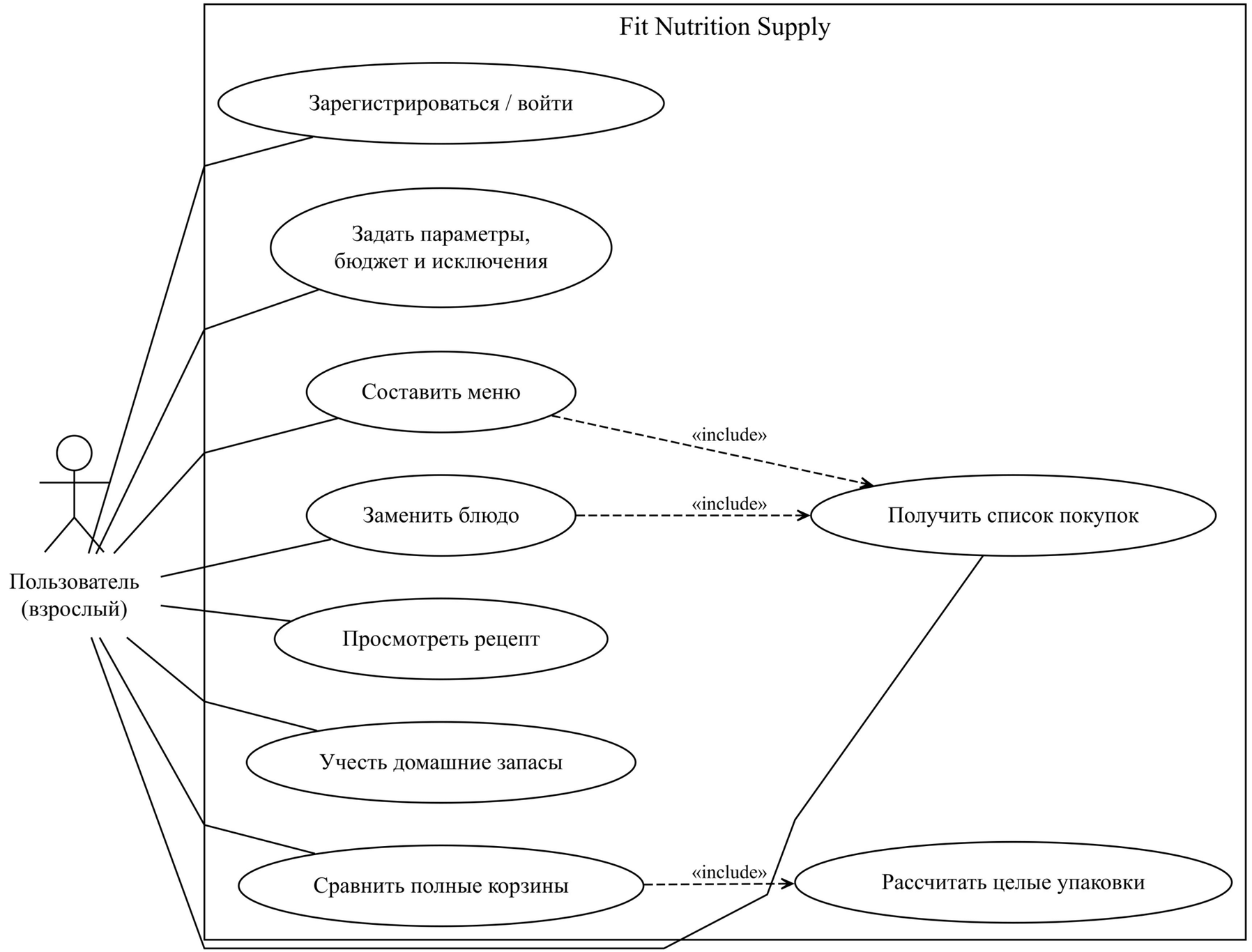


Рисунок 2 - Диаграмма вариантов использования

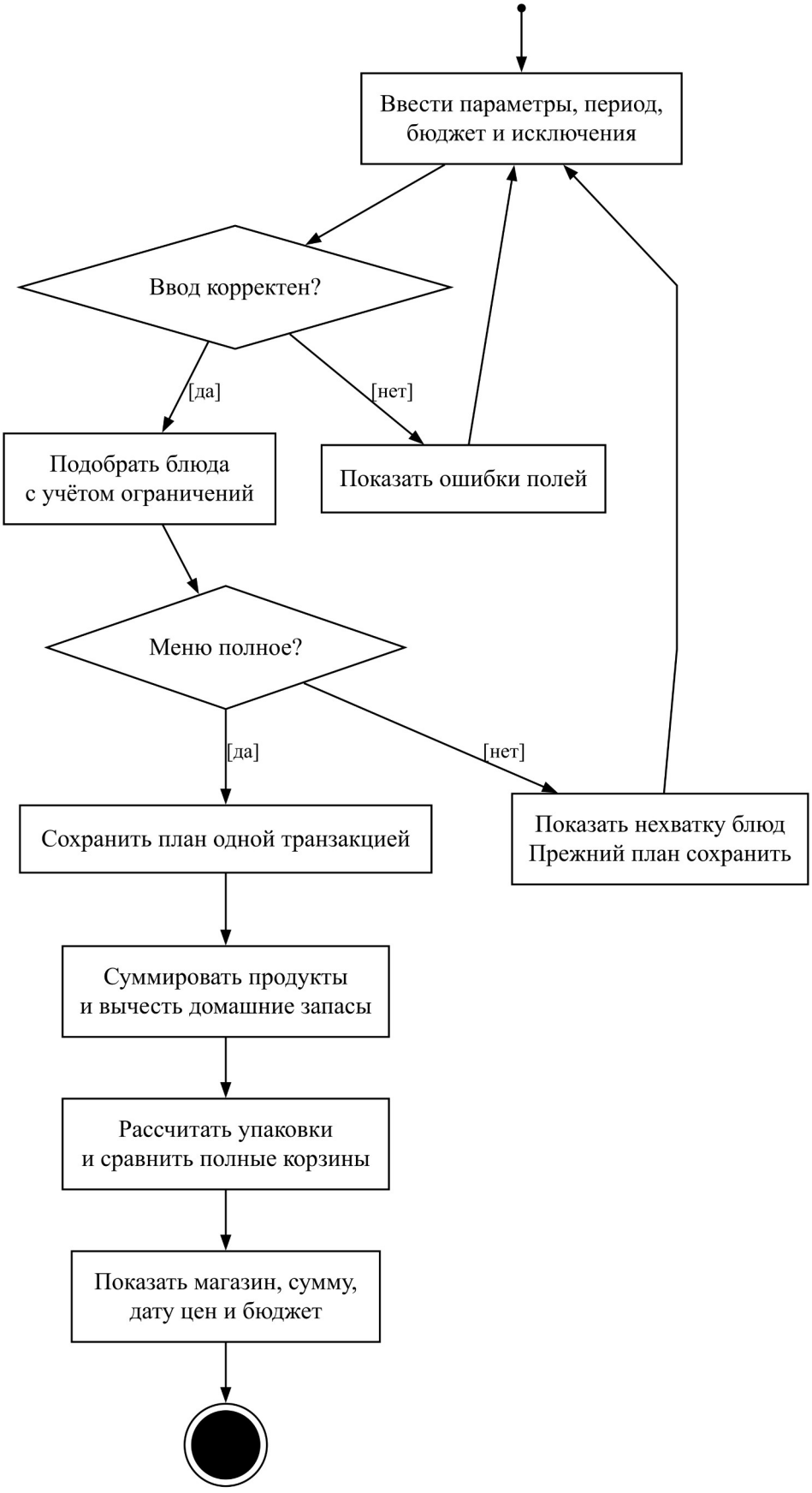


Рисунок 3 - Основной сценарий

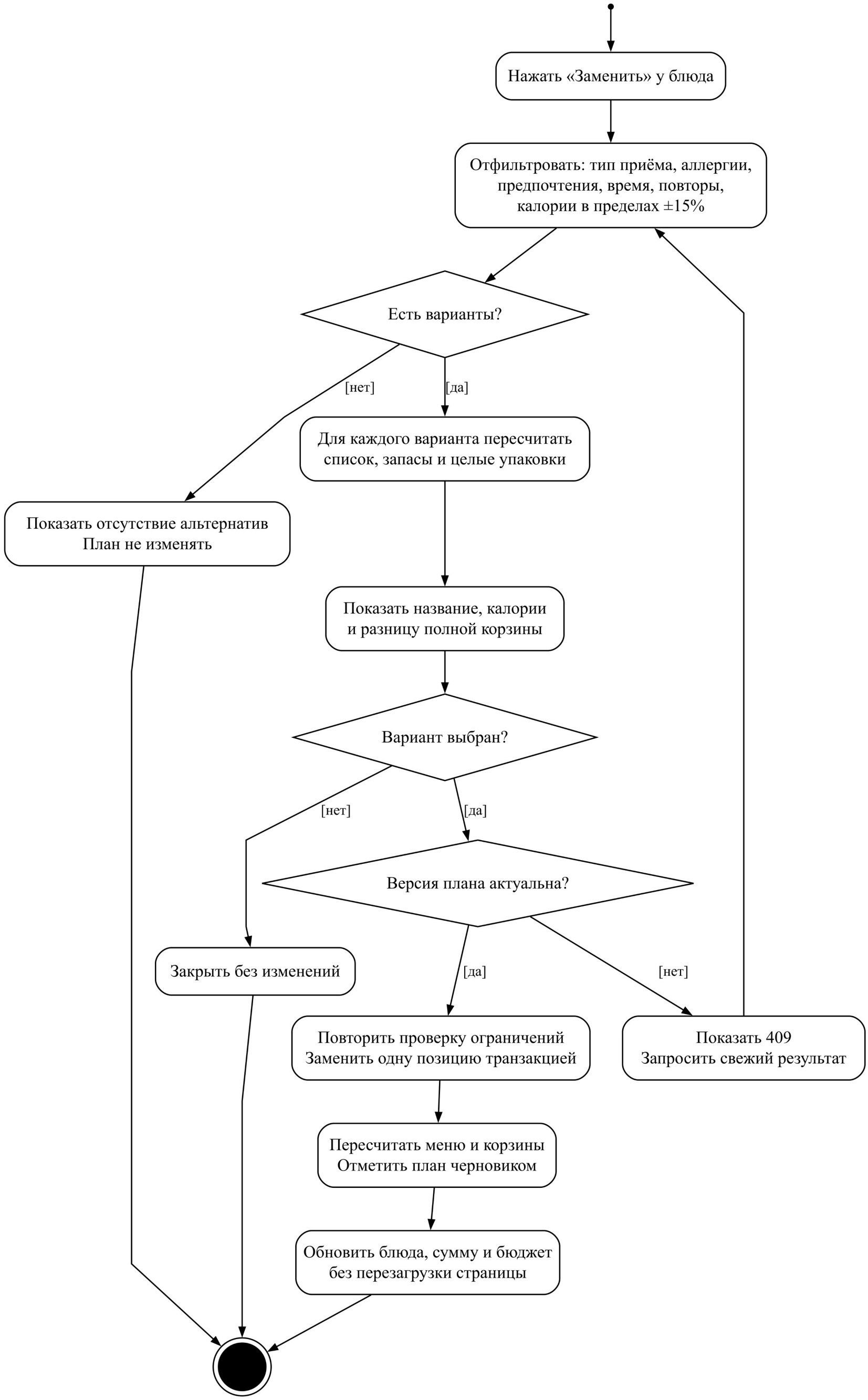


Рисунок 4 - Альтернативный сценарий замены блюда

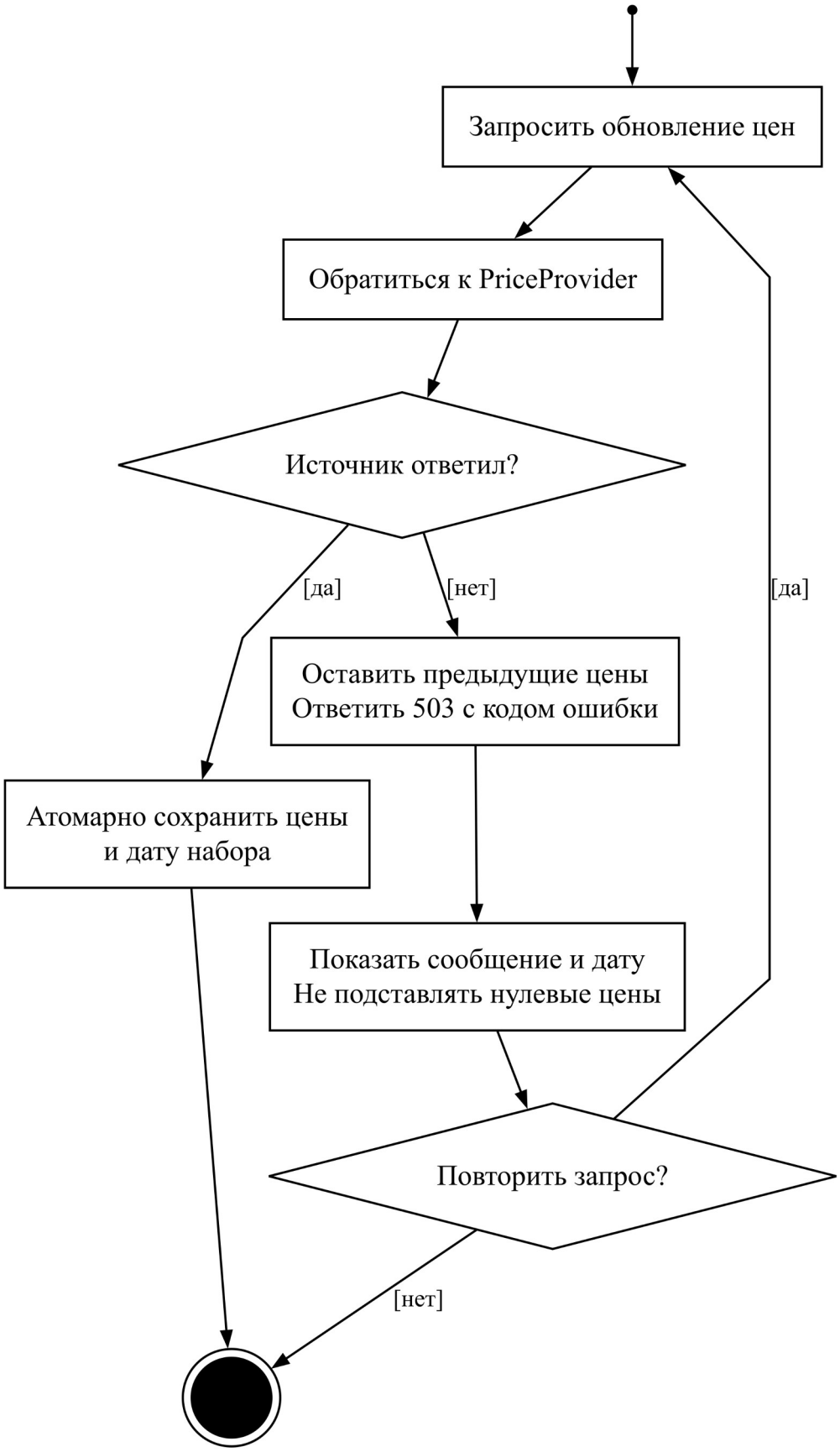


Рисунок 5 - Сценарий ошибки источника цен

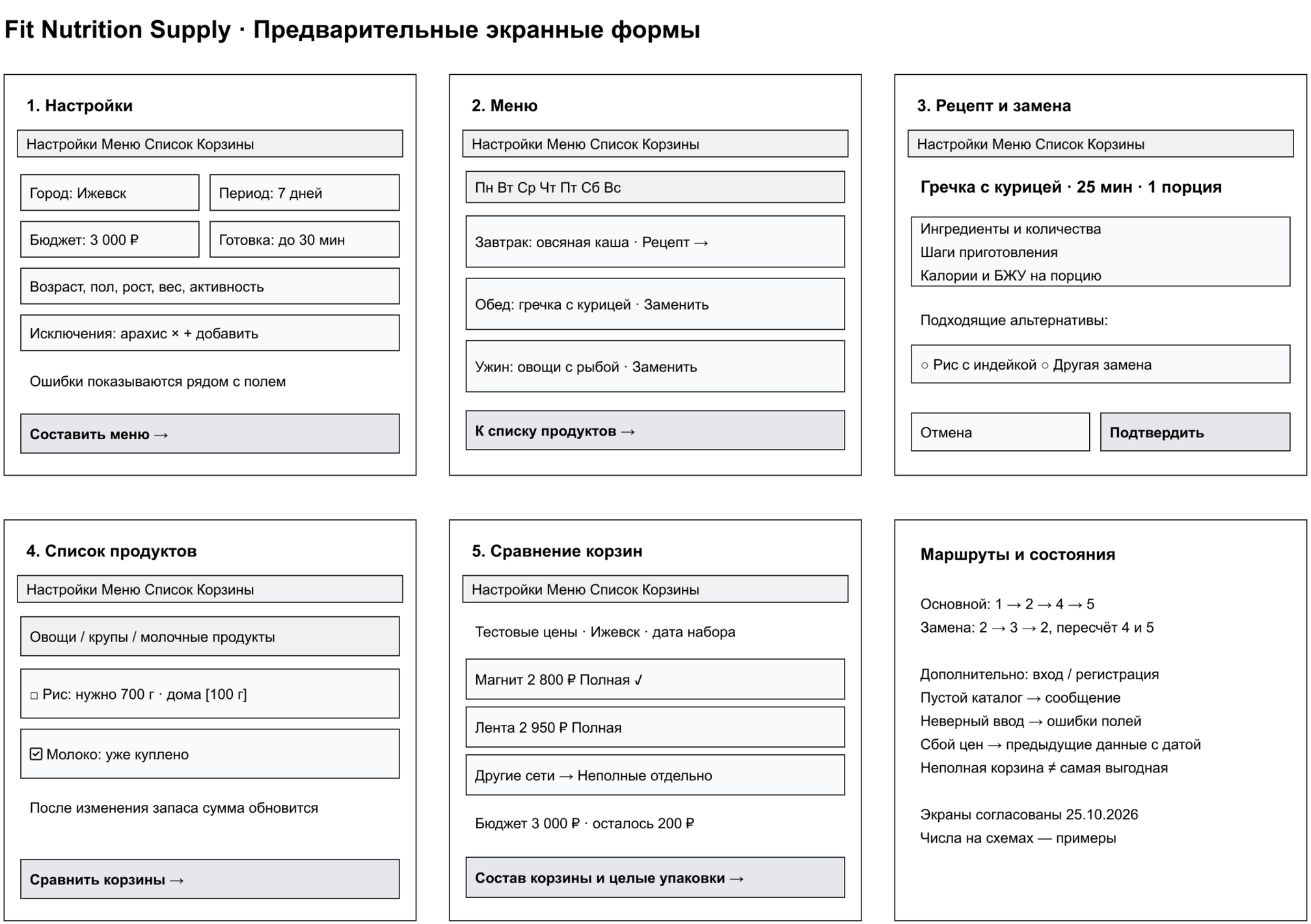


Рисунок 6 - Предварительные экранные формы

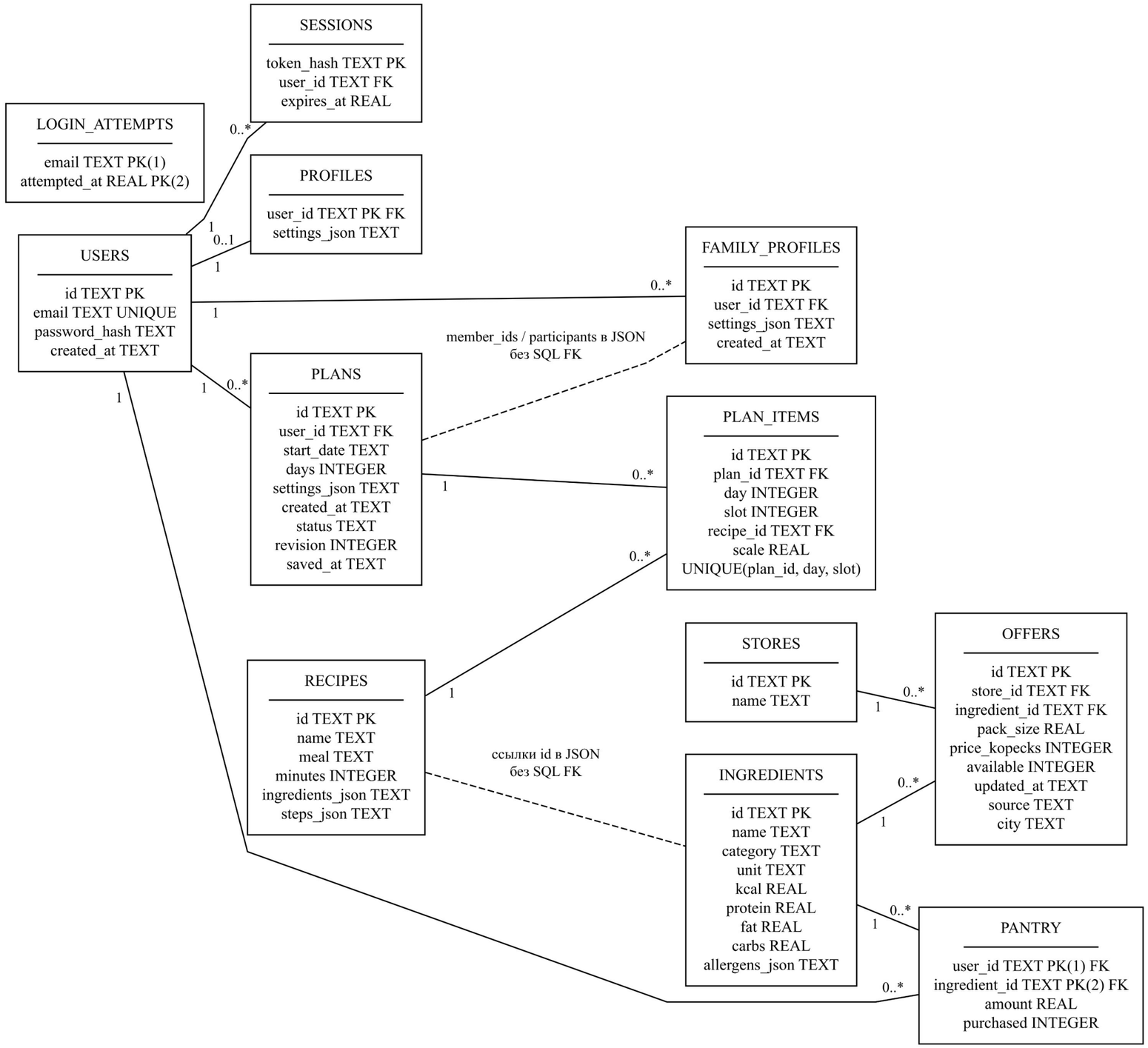
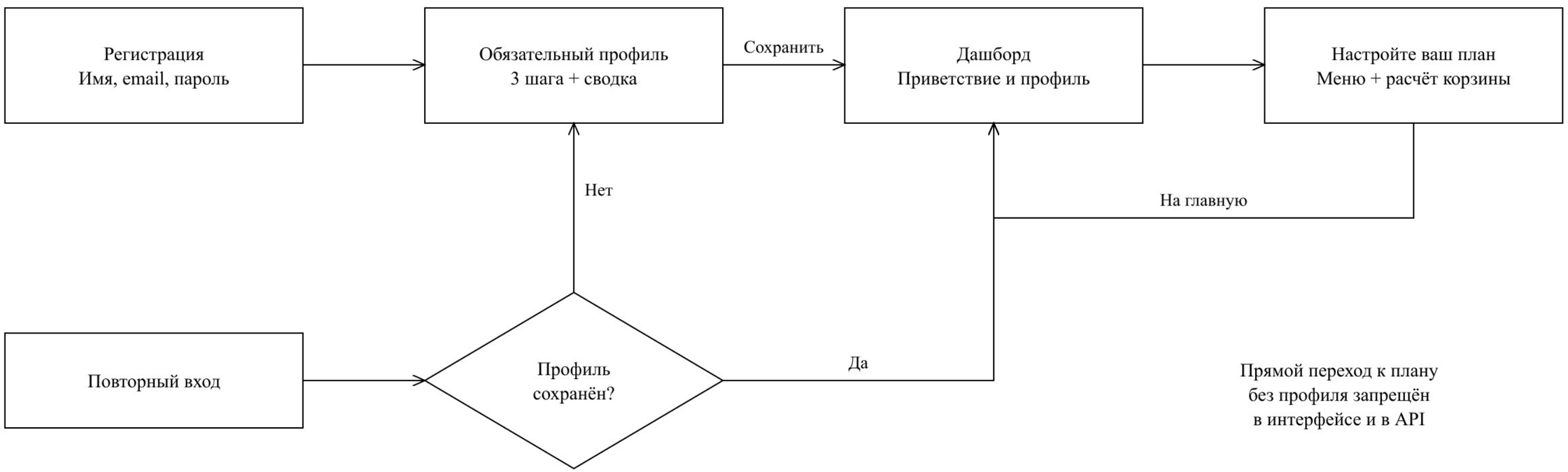


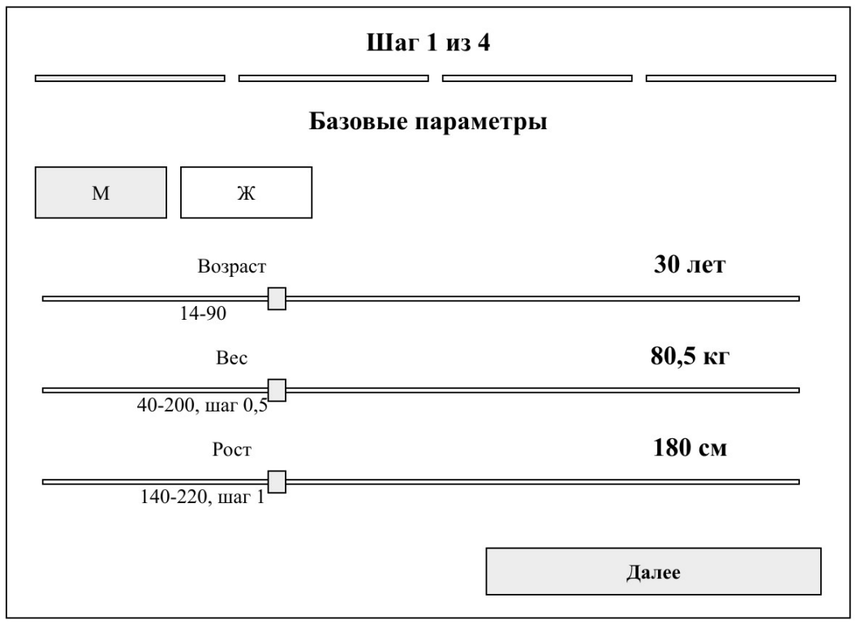
Рисунок 7 - ER-модель прототипа



Профиль хранится отдельно в SQLite. Черновик шагов хранится в браузере по аккаунту. Возврат и перезагрузка не стирают черновик. План сохраняет снимок параметров.

Рисунок 8 - Поток регистрации, профиля, дашборда и плана

Fit Nutrition Supply: онбординг профиля





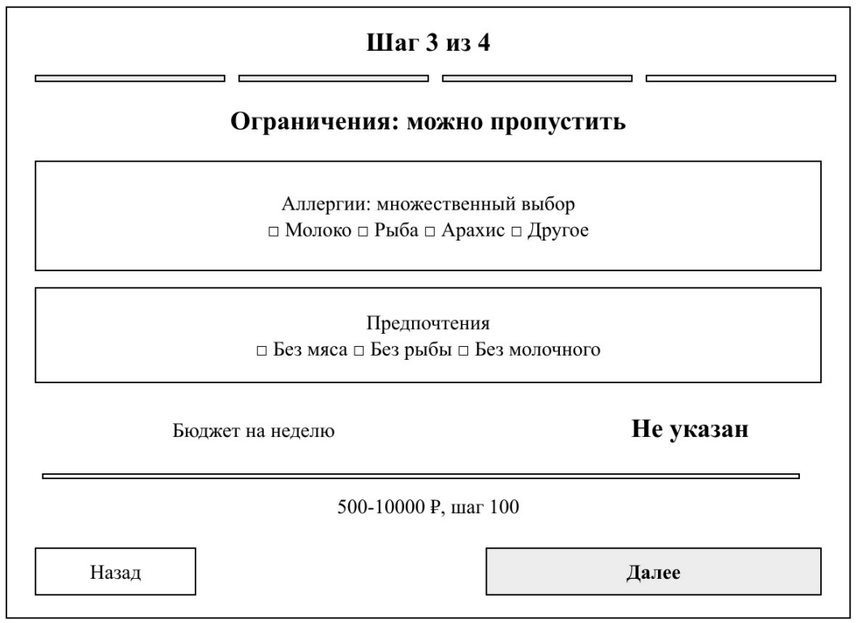




Рисунок 9 - Четыре экрана онбординга

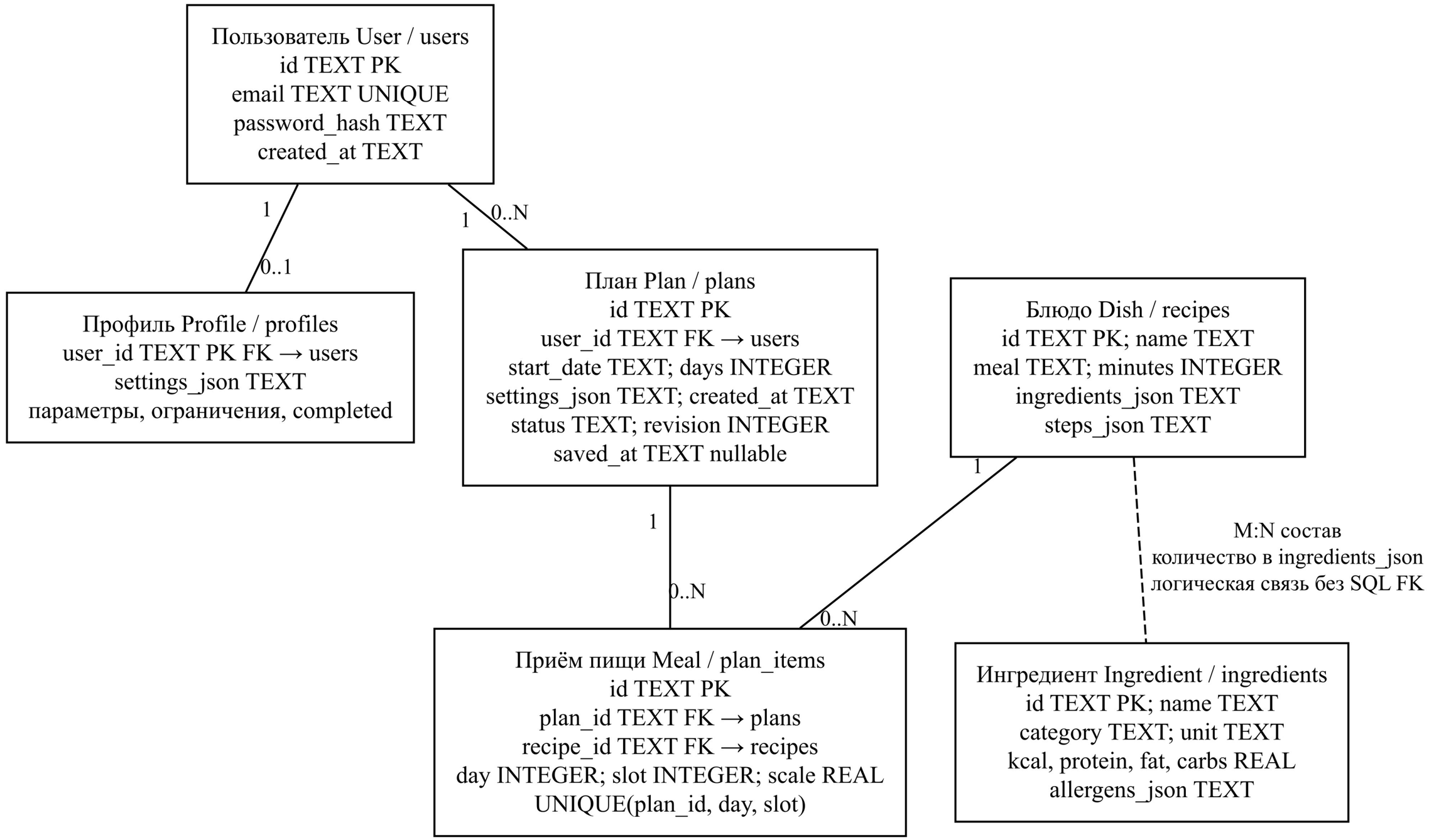


Рисунок 10 - Предметная ER-модель и соответствие SQLite

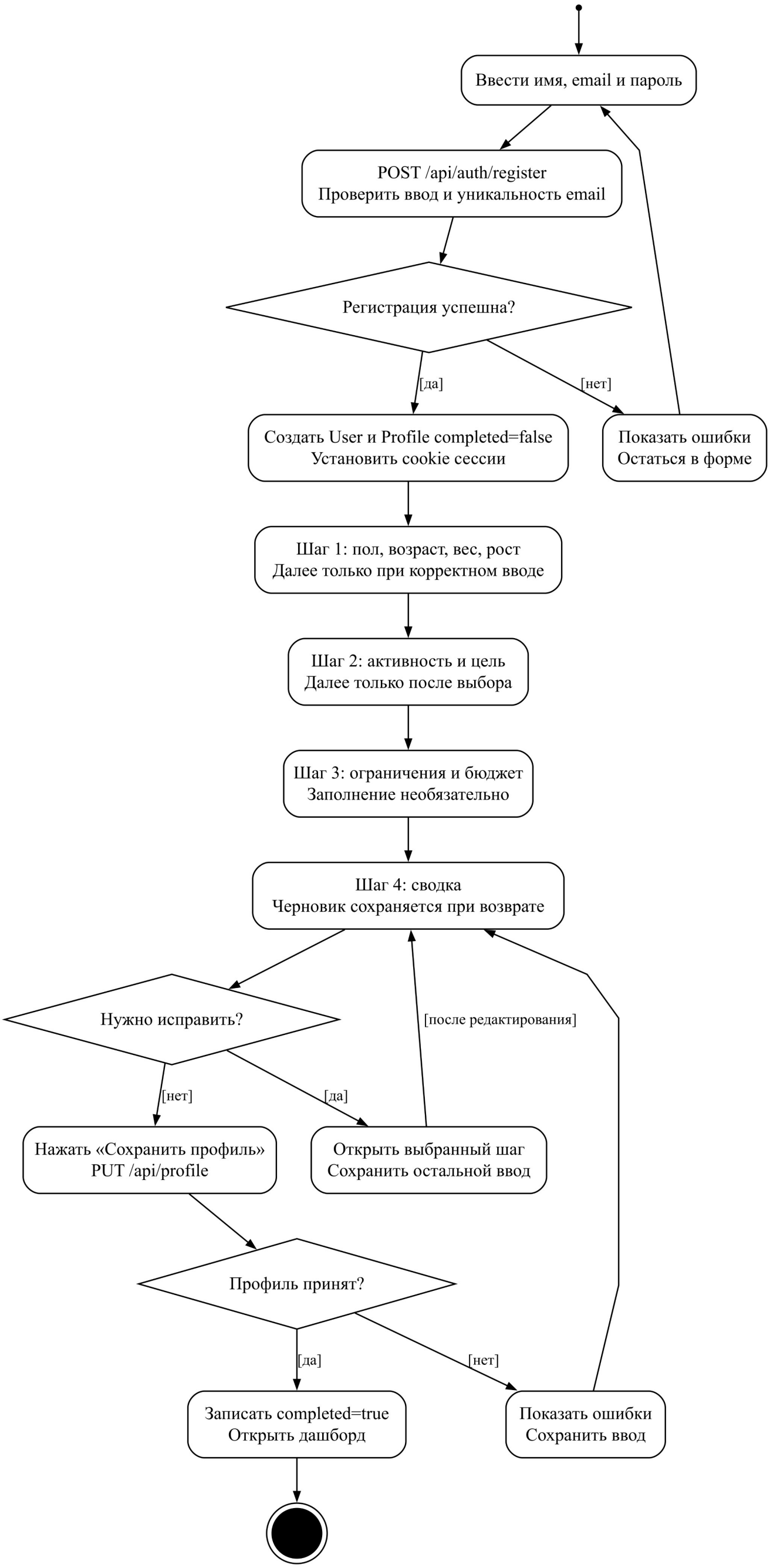


Рисунок 11 - Регистрация и заполнение профиля

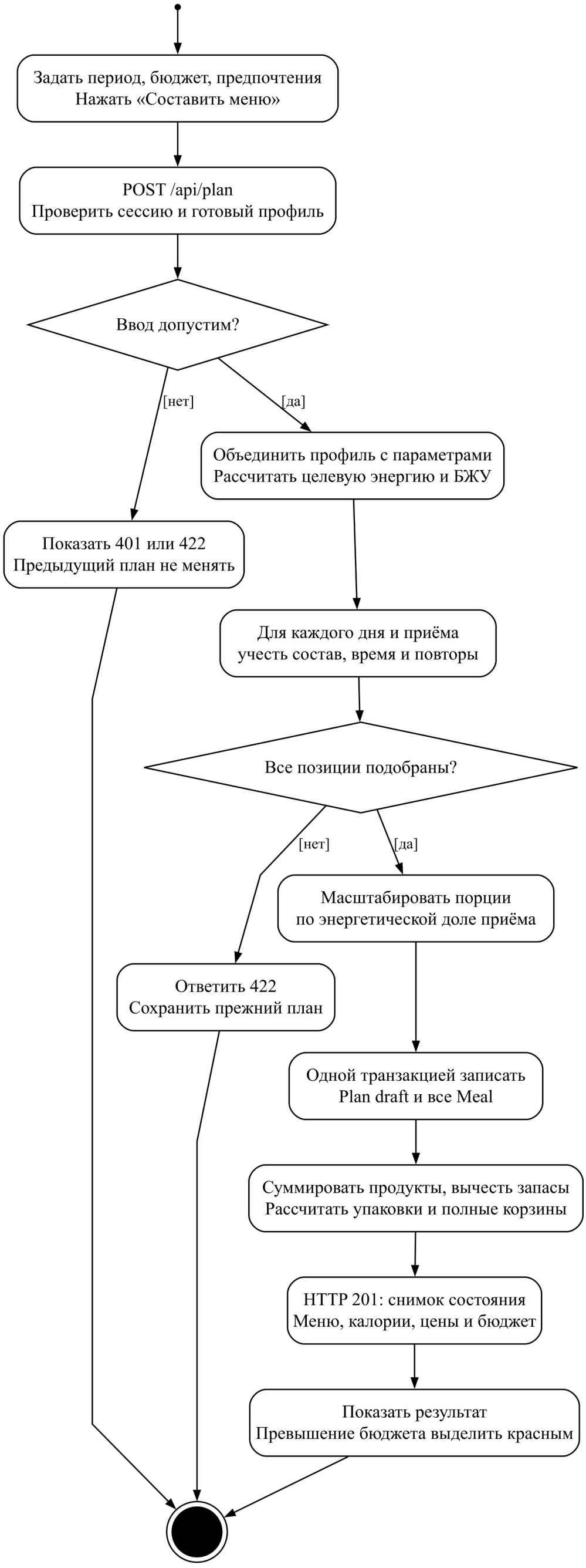


Рисунок 12 - Генерация меню

ПРИЛОЖЕНИЕ Б ПРОТОКОЛ ПРОВЕРКИ

Ниже сохранён протокол первоначального прогона от 26.10.2026. Повторная проверка от 30.10.2026, включая 31 тест и семейные сценарии, описана в разделе 9 отчёта.

Дата: 26.10.2026. Среда: Windows 11 Pro, версия 10.0.26200; Python 3.12.14 и дополнительный прогон Python 3.10; Google Chrome 155.0.8059.85. Браузер управляется Playwright, приложение запущено на 127.0.0.1. Тестовые аккаунты и временные базы не содержат пользовательских данных.

Расчёты и HTTP API

Команда: python -m unittest discover -s prototype/tests -v из корня репозитория после установки prototype/requirements.txt.

Результат: 24 теста пройдены, ошибок и пропусков нет. Проверены 9 доменных и 15 API-сценариев. API-тесты поднимают отдельный сервер на свободном порту и используют временную SQLite.

Таблица 1

Группа	Проверка	Результат
Нормы	Пример ЛР1: 2345 ккал, 176/78/235 г БЖУ	Пройдено
Валидация	6/32 дня, дробный период, нулевой бюджет, NaN	Отклонены
План	7 дней × 3 приёма, энергия дня	21 позиция, отклонение менее 1 ккал на учебных данных
Длинный период	31 день × 5 приёмов, лимит повторов	155 позиций, не более 2 повторов в календарной неделе
Исключения	Арахис и орехи не попадают в план	Пройдено
Нехватка блюд	Ограничение 5 минут, невозможный основной приём	Явная ошибка без снятия ограничений
Упаковки	700 г нужно, 100 г дома, пачка 500 г за 100 ₽	2 пачки, 200 ₽, остаток 400 г
Полнота	Дешёвая неполная корзина рядом с полной	Неполная не рекомендуется
Экономия	Корзины 1250/1180/1210 ₽	Экономия 30 ₽ со следующей полной
Авторизация	Регистрация, повтор email, bcrypt cost 12, вход второй сессии	Пройдено
Ограничение попыток	5 неверных паролей и следующая попытка	Следующая попытка: 429
Изоляция	Запрос позиции чужого плана	404; чужой план не раскрыт
Выход	Повторное чтение state после logout	401

Группа	Проверка	Результат
Отказ источника	503 при обновлении	Цены и суммы неизменны
Ошибка плана	Неверный период и нехватка блюд	Предыдущий план сохранён
Origin	Изменяющий запрос с внешнего домена	403

Браузерный сценарий

Файл проверки: prototype/tests/browser.cjs. Машинный результат: evidence/browser-results.json. Пройдено: демонстрационный вход; заполнение формы; генерация; замена с подтверждением; учёт запаса; пять корзин; ошибка источника; перезагрузка с сохранением плана; отказ чтения состояния без сессии. Ошибок JavaScript в прогоне нет.

Таблица 2

Браузер	Настройки	Меню	Список	Корзины
Chrome 155.0.8059.85, 360 px	Пройдено	Пройдено	Пройдено	Пройдено
Chrome 155.0.8059.85, 768 px	Пройдено	Пройдено	Пройдено	Пройдено
Chrome 155.0.8059.85, 1920 px	Пройдено	Пройдено	Пройдено	Пройдено
Firefox 158.0, те же ширины	Не проверено	Не проверено	Не проверено	Не проверено
Safari 26.0, те же ширины	Не проверено	Не проверено	Не проверено	Не проверено

Горизонтальная прокрутка документа отсутствует на всех 12 проверенных сочетаниях. Компоновка проверяется в настольном браузере; реальная сенсорная клавиатура, устройства iOS/Android и Safari в этом окружении не проверялись.

Границы результатов

Result.cjs: прогноз цены, отмена, бюджет, сохранение и замена без перезагрузки проверены на 360/768/1920 px. Онбординг: 18 сочетаний без переполнения, возврат и восстановление черновика. Протоколы: evidence/result-results.json и evidence/onboarding-results.json.

NFR-02/04, реальные обновления NFR-03, Firefox и Safari не проверены. Каталог учебный, результаты не подтверждают рыночную экономию или медицинскую пригодность.